PROJECT REPORT
Project no: 14

Project title:

Design and Development of a GPS-Based Vehicle Tracking and Fleet Management System

Submitted by:
Arundeepak S – 24MER004
DeepakP – 24MER006
Dharanidharan R S – 24MER007
Dinesh k – 24MER009
Vishnuvarthan P -24MER063

Design and Development of a GPS-Based Vehicle Tracking and Fleet Management System

1. Project Overview

This project presents the design and development of a GPS-Based Vehicle Tracking and Fleet Management System built around the ESP32 microcontroller. The system continuously acquires the geographic location, speed, and satellite-fix quality of a vehicle using a GPS receiver module, and transmits this data wirelessly over Wi-Fi to a cloud-based IoT platform (Adafruit IO) using the MQTT protocol. Fleet operators can then view the live position of the vehicle on an interactive map dashboard, along with numeric readouts of speed and the number of satellites currently in view.

The prototype demonstrates the complete pipeline of a modern telematics solution — sensing, edge processing, wireless communication, cloud ingestion, and visualization — using low-cost, readily available hardware. It is intended as a proof-of-concept for applications such as logistics fleet monitoring, school bus tracking, delivery vehicle management, and personal vehicle anti-theft tracking.

Objectives

- Continuously determine vehicle location (latitude/longitude), speed, and satellite count using a GPS module.
- Transmit sensor data to a cloud dashboard in near real time using Wi-Fi and MQTT.
- Visualize vehicle position on a live map and key parameters on numeric/gauge widgets.
- Build a low-power, compact, and cost-effective hardware prototype suitable for in-vehicle deployment.
- Evaluate the accuracy, update rate, and reliability of the tracking system under real-world conditions.

2. Problem Statement

Conventional fleet management in small and medium transportation businesses is often carried out manually — through phone calls, driver logs, or periodic check-ins — which is slow, error-prone, and offers no real-time visibility into vehicle location or movement. Without live tracking, fleet operators cannot verify routes, respond quickly to delays or breakdowns, optimize delivery schedules, or ensure the safety and security of vehicles and drivers.

Commercial GPS fleet-tracking solutions exist, but many require expensive proprietary hardware and recurring subscription costs, which places them out of reach for small operators, individual vehicle owners, or educational prototyping. There is a clear need for an affordable, open, and easily extensible tracking solution that can report a vehicle's live location and motion parameters to a remote dashboard accessible from anywhere with an internet connection.

3. Proposed Solution

The proposed solution uses an ESP32 development board as the central controller, interfaced with a NEO-6M GPS receiver module over a hardware UART (Serial2) port. The ESP32 parses raw NMEA GPS sentences using the TinyGPS++ library to extract latitude, longitude, speed, and the number of visible satellites.

Once a valid GPS fix is obtained, the ESP32 connects to a local Wi-Fi network and publishes the extracted data to three separate feeds — vehicle-location, vehicle-speed, and vehicle-sats — on the Adafruit IO cloud platform using the lightweight MQTT publish/subscribe protocol. Adafruit IO's dashboard feature is used to build a live "GPS Monitoring" dashboard, which plots the vehicle's position on an interactive street map and displays speed and satellite count on gauge widgets, refreshed automatically as new data arrives.

This architecture keeps the on-vehicle hardware minimal (microcontroller + GPS module + power supply) while offloading storage, visualization, and remote accessibility to the cloud, so the dashboard can be viewed from any browser or device without additional infrastructure.

4. System Architecture

The system follows a simple three-layer IoT architecture: a Sensing Layer (GPS module), a Processing & Communication Layer (ESP32 + Wi-Fi + MQTT), and a Cloud/Visualization Layer (Adafruit IO dashboard).

Architecture Flow

- GPS Module (NEO-6M): Receives satellite signals and outputs NMEA sentences over its TX pin.
- ESP32 (UART2): Reads the raw NMEA stream on GPIO16 (RX2) and decodes it using TinyGPS++.
- ESP32 (Wi-Fi + MQTT Client): Connects to the wireless network and publishes decoded values to Adafruit IO over MQTT (port 1883).
- Adafruit IO Cloud: Receives and stores feed data, and renders it on a live dashboard (map, gauges).
- End User: Views vehicle location, speed, and satellite status remotely through any web browser.

5. Circuit Table

The table below lists the physical wiring between the NEO-6M GPS module and the ESP32 development board, as implemented in the prototype and shown in the wiring diagram above.

GPS Module Pin	ESP32 Pin	Function
VCC	3V3	3.3 V power supply to the GPS module
GND	GND	Common ground reference
TX	GPIO16 (RX2 / UART2)	GPS transmits NMEA data to the ESP32
RX	GPIO17 (TX2 / UART2)	ESP32 transmits configuration data to the GPS (unused in normal operation)

Power for the assembled prototype is supplied over USB from a laptop/adapter during testing; for standalone in-vehicle operation, the Bill of Materials includes a Li-ion battery with a TP4056 charging/protection module.

6. Circuit Design

The circuit was first laid out and verified in a schematic/wiring tool (Figure 4.1) before being physically assembled on a breadboard, as shown below. The GPS module's VCC, GND, TX, and RX lines are routed to the ESP32's 3V3, GND, and UART2 pins respectively, keeping the wiring minimal since the module handles NMEA decoding-independent signal generation internally.

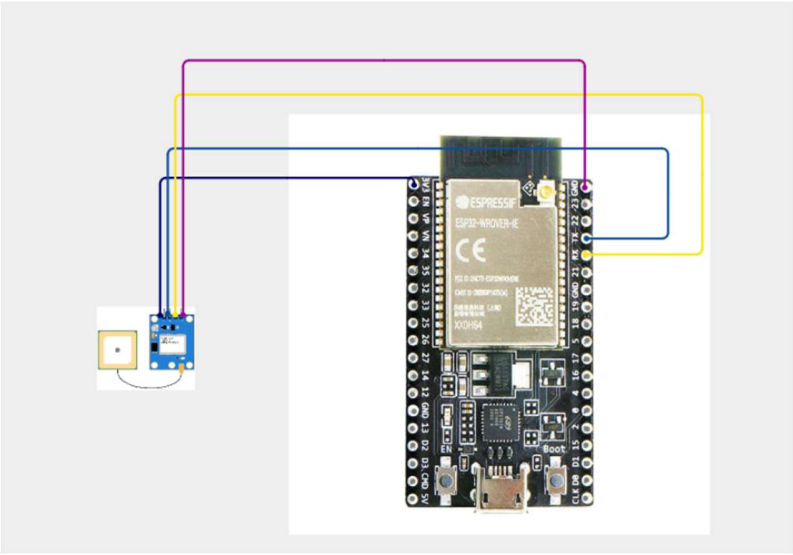

Figure 1 — Wiring/connection diagram of the NEO-6M GPS module to the ESP32-WROVER-IE development board.

Design Considerations

- Serial2 (UART2) is used for GPS communication so that Serial0 (USB) remains free for debugging output via the Serial Monitor.
- The GPS module is placed with a clear line of sight (active ceramic antenna) to improve satellite acquisition time.
- A common ground between the GPS module and ESP32 is essential for reliable UART communication.
- Decoupling and short wiring runs are used to minimize noise on the 3.3 V rail feeding the GPS module.

7. Algorithm

The firmware logic running on the ESP32 can be summarized as follows:

1. Initialize Serial (115200 baud) for debugging and Serial2 (9600 baud) on GPIO16/17 for GPS communication.
2. Connect to the configured Wi-Fi network (WLAN_SSID / WLAN_PASS) and wait until a connection is established.
3. In the main loop, continuously read any available bytes from the GPS serial port and feed them to the TinyGPS++ parser (gps.encode()).
4. Check and, if necessary, (re)establish the MQTT connection to the Adafruit IO broker (MQTT_connect()).
5. Every 15 seconds (publishInterval), check whether the GPS has processed a meaningful number of characters; if not, log a warning that no GPS data is being received.
6. If a valid GPS location fix is available: extract latitude, longitude, speed (km/h), and satellite count.
7. Format the latitude/longitude into a comma-separated "lat,lng,elevation" string as required by Adafruit IO's location feed format.
8. Publish the location string, speed value, and satellite count to their respective Adafruit IO feeds (vehicle-location, vehicle-speed, vehicle-sats) over MQTT.
9. If no valid fix is available yet, print the current satellite-in-view count to the Serial Monitor and continue searching.
10. Repeat the loop indefinitely, re-acquiring GPS data and publishing on each 15-second interval.

8. Coding

The complete embedded firmware, written in C++ for the Arduino/ESP32 framework, is listed below. It uses the TinyGPS++ library to parse GPS data and the Adafruit MQTT Client library to publish data to Adafruit IO.

```
#include <WiFi.h>
#include "Adafruit_MQTT.h"
#include "Adafruit_MQTT_Client.h"
#include <TinyGPS++.h>
```

```
#include <HardwareSerial.h>

// ===== WiFi credentials =====
#define WLAN_SSID       "test"
#define WLAN_PASS       "test1234"

// ===== Adafruit IO credentials =====
#define AIO_SERVER      "io.adafruit.com"
#define AIO_SERVERPORT  1883
#define AIO_USERNAME    "dharanidharan_97969"
#define AIO_KEY         "aio_xxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxx"

// ===== GPS setup =====
TinyGPSPlus gps;
HardwareSerial gpsSerial(2); // UART2
#define GPS_RX 16
#define GPS_TX 17
#define GPS_BAUD 9600

// ===== MQTT client setup =====
WiFiClient client;
Adafruit_MQTT_Client mqtt(&client, AIO_SERVER, AIO_SERVERPORT, AIO_USERNAME, AIO_KEY);

// ===== Feeds =====
Adafruit_MQTT_Publish locationFeed = Adafruit_MQTT_Publish(&mqtt, AIO_USERNAME "/feeds/vehicle-location");
Adafruit_MQTT_Publish speedFeed    = Adafruit_MQTT_Publish(&mqtt, AIO_USERNAME "/feeds/vehicle-speed");
Adafruit_MQTT_Publish satFeed      = Adafruit_MQTT_Publish(&mqtt, AIO_USERNAME "/feeds/vehicle-sats");

unsigned long lastPublish = 0;
const unsigned long publishInterval = 15000; // 15 sec

void MQTT_connect();

void setup() {
  Serial.begin(115200);
  gpsSerial.begin(GPS_BAUD, SERIAL_8N1, GPS_RX, GPS_TX);

  Serial.println("\n--- ESP32 GPS Vehicle Tracker ---");

  // Connect WiFi
  Serial.print("Connecting to WiFi");
  WiFi.begin(WLAN_SSID, WLAN_PASS);
  while (WiFi.status() != WL_CONNECTED) {
    delay(500);
    Serial.print(".");
  }
  Serial.println();
  Serial.print("WiFi connected. IP: ");
```

```
  Serial.println(WiFi.localIP());
}

void loop() {
  // Feed GPS parser
  while (gpsSerial.available() > 0) {
    char c = gpsSerial.read();
    gps.encode(c);
  }

  MQTT_connect();

  // Publish every publishInterval ms
  if (millis() - lastPublish > publishInterval) {
    lastPublish = millis();

    Serial.println("----------------------------------------");
    Serial.print("Bytes Processed : ");
    Serial.println(gps.charsProcessed());

    if (gps.charsProcessed() < 10) {
      Serial.println("WARNING: No data received from GPS module!");
      return;
    }

    if (gps.location.isValid()) {
      double lat = gps.location.lat();
      double lng = gps.location.lng();
      double speedKmph = gps.speed.kmph();
      int sats = gps.satellites.value();

      Serial.print("Latitude  : "); Serial.println(lat, 6);
      Serial.print("Longitude : "); Serial.println(lng, 6);
      Serial.print("Speed     : "); Serial.print(speedKmph, 1); Serial.println(" km/h");
      Serial.print("Satellites: "); Serial.println(sats);

      // Build GPS-format string: lat,lng,elevation
      char locationStr[64];
      snprintf(locationStr, sizeof(locationStr), "%.6f,%.6f,0", lat, lng);

      // Publish to Adafruit IO
      if (!locationFeed.publish(locationStr)) {
        Serial.println("Failed to publish location");
      } else {
        Serial.println("Location published.");
      }

      if (!speedFeed.publish((float)speedKmph)) {
        Serial.println("Failed to publish speed");
      }

      if (!satFeed.publish((int32_t)sats)) {
```

```
      Serial.println("Failed to publish satellite count");
     }

    } else {
     Serial.print("Status    : Searching for satellite fix...");
     Serial.print(" (Satellites in view: ");
     Serial.print(gps.satellites.value());
     Serial.println(")");
    }
  }
}

// Connects/reconnects to Adafruit IO MQTT broker
void MQTT_connect() {
  int8_t ret;

  if (mqtt.connected()) {
    return;
  }

  Serial.print("Connecting to MQTT... ");

  uint8_t retries = 3;
  while ((ret = mqtt.connect()) != 0) {
    Serial.println(mqtt.connectErrorString(ret));
    Serial.println("Retrying MQTT connection in 5 seconds...");
    mqtt.disconnect();
    delay(5000);
    retries--;
    if (retries == 0) {
      Serial.println("Giving up on MQTT connection for now.");
      return;
    }
  }

  Serial.println("MQTT Connected!");
}
```

Key Libraries Used

- WiFi.h — manages the ESP32's Wi-Fi station connection.
- Adafruit_MQTT / Adafruit_MQTT_Client — implements the MQTT publish client used to talk to Adafruit IO.
- TinyGPS++ — parses raw NMEA sentences into usable latitude, longitude, speed, and satellite data.
- HardwareSerial — provides access to the ESP32's UART2 peripheral for the GPS interface.

9. System Working

On power-up, the ESP32 initializes its serial interfaces and connects to the configured Wi-Fi network, printing its assigned IP address to the Serial Monitor for diagnostic purposes. It then enters its main loop, where it continuously streams bytes from the GPS module into the TinyGPS++ decoder.

The GPS module searches for satellite signals; once enough satellites are acquired, it produces a valid position fix (visible in testing as latitude ≈ 11.27° N, longitude ≈ 77.60° E, matching the Perundurai / Coimbatore region). Every 15 seconds, the ESP32 checks the MQTT connection status, reconnecting if necessary, and publishes the current latitude/longitude, speed, and satellite count to their respective Adafruit IO feeds.

On the cloud side, the Adafruit IO "GPS Monitoring" dashboard subscribes to these feeds and updates its widgets automatically as new values are published: the map widget re-plots the vehicle-location marker at the new coordinates, while the speed and satellite-count gauges update their numeric readouts. This gives a fleet operator a continuously refreshed, remote view of the vehicle's status without needing any custom server infrastructure.

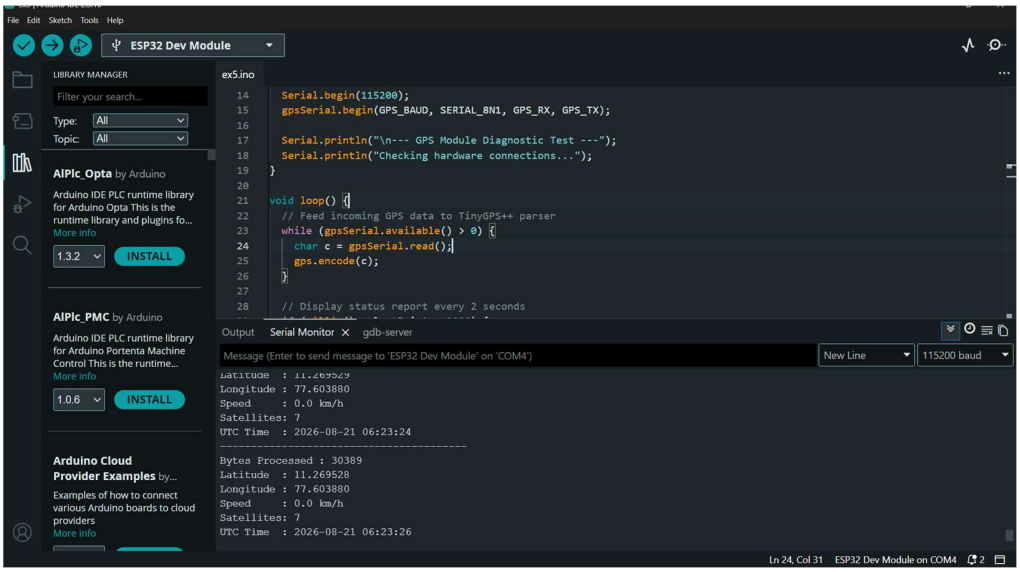

Figure2 — Arduino IDE Serial Monitor showing successful GPS diagnostic output: latitude, longitude, speed, satellite count, and UTC time.

10. Bill of Materials

Component	Quantity	Purpose
ESP32 Development Board (WROOM/WROVER)	1	Main microcontroller — Wi-Fi, MQTT client, GPS parsing
NEO-6M GPS Module (with active antenna)	1	Acquires satellite location, speed, and time data
OLED Display	1	Optional local status/diagnostic display
Li-ion Battery	1	Portable power source for in-vehicle operation
TP4056 Charging Module	1	Battery charging and protection circuit
Breadboard	1	Prototyping platform for circuit assembly
Jumper Wires	1 set	Interconnections between GPS module and ESP32
Micro-USB Cable	1	Programming and power supply during development/testing

11. Prototype Implementation

The prototype was assembled on a standard breadboard for development and testing. The ESP32 development board is mounted at the bottom of the breadboard and connected via a micro-USB cable to a laptop for power and programming. The NEO-6M GPS module is mounted above it, with its active antenna positioned separately for a clearer view of the sky, and connected to the ESP32's UART2 pins using color-coded jumper wires (VCC, GND, TX, RX).

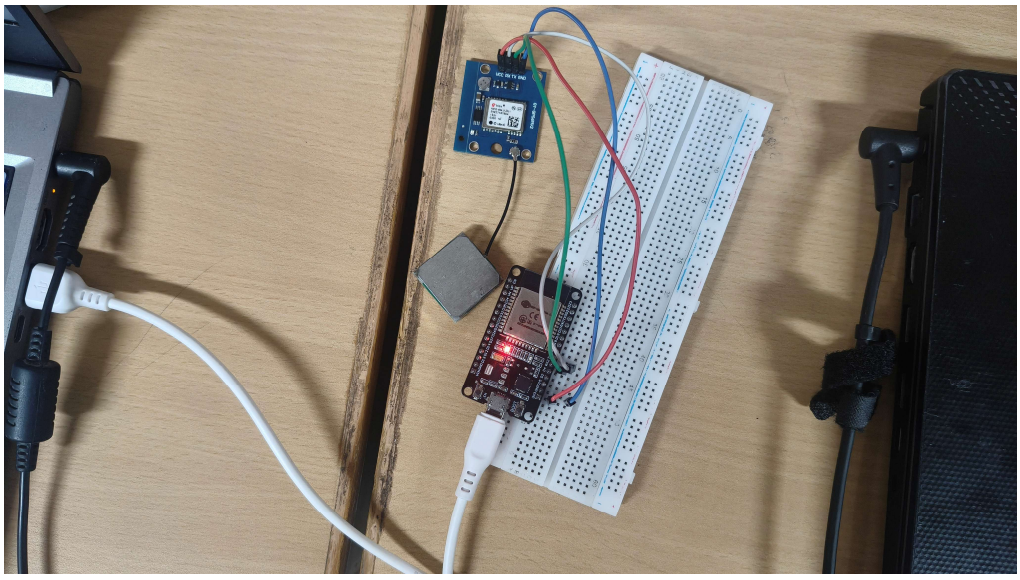

Figure 3 — Assembled prototype: ESP32 board, NEO-6M GPS receiver, and antenna, wired on a breadboard and powered via USB.

During testing, the Arduino IDE Serial Monitor was used to verify that the GPS module was acquiring a valid fix and that the parsed values (latitude, longitude, speed, satellite count, and UTC time) were being printed correctly before being published to the cloud.

12. Results & Performance Analysis

The completed system was tested by running the firmware continuously and observing both the local Serial Monitor output and the remote Adafruit IO "GPS Monitoring" dashboard. The results confirm that the end-to-end pipeline — GPS acquisition, NMEA parsing, MQTT publishing, and cloud visualization — functions correctly.

Serial Diagnostic Output

The Serial Monitor log confirmed a stable satellite fix with 7 satellites in view, and consistent latitude/longitude values (≈ 11.2695° N, 77.6038° E) across successive readings, with the GPS module correctly reporting 0.0 km/h speed for a stationary test unit and a correctly time-stamped UTC time field, confirming that the module's fix was valid and current.

Cloud Dashboard Behaviour

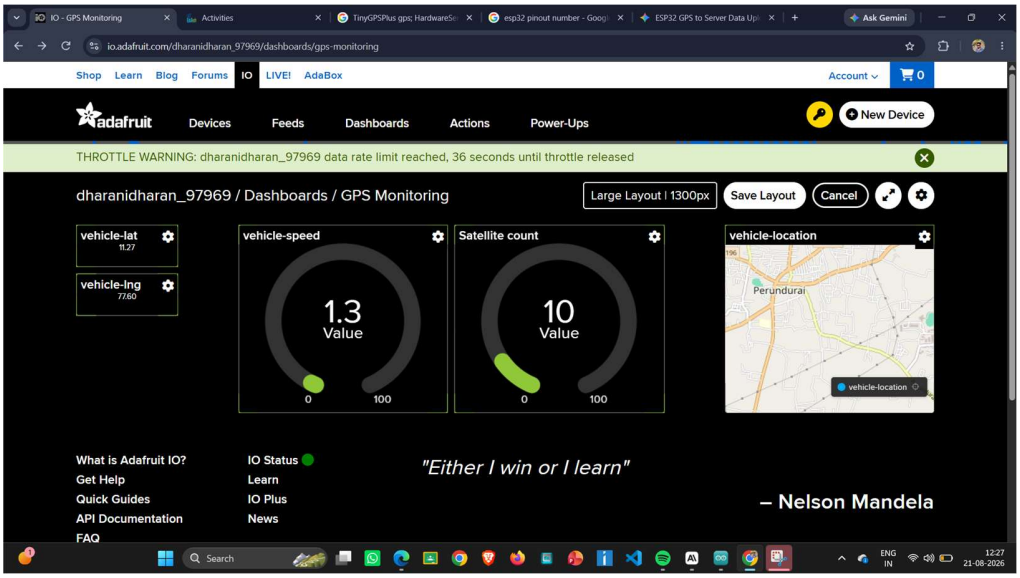

Figure 4 — Adafruit IO dashboard showing vehicle-lat (11.27), vehicle-lng (77.60), speed, satellite count, and the vehicle-location map centered on Perundurai — matching the physical test location.

As shown above, the location feed correctly plotted the vehicle marker over Perundurai, near Coimbatore, matching the real-world test location of the prototype, and the satellite-count gauge read 10 during a period of strong sky visibility.

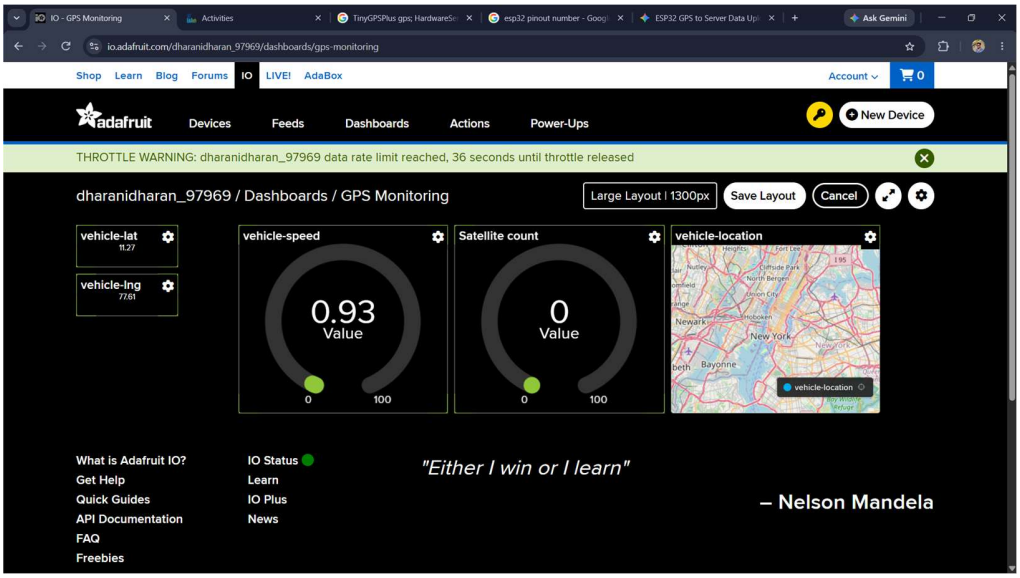

Figure 5 — Dashboard block-configuration view for the vehicle-location map widget, confirming the marker plots correctly on the OpenStreetMap base layer.

One operational observation during testing was a recurring "THROTTLE WARNING" banner on the Adafruit IO dashboard, indicating that the free-tier data rate limit had been reached. This occurred because the publish interval (15 seconds across three feeds) approached Adafruit IO's request-rate ceiling; the system recovered automatically once the throttle window expired, and no data was permanently lost, but this highlights the need to tune the publish interval (or batch feed updates) for sustained deployment.

Performance Summary

Parameter	Observed Result
Satellite acquisition (cold start)	Achieved a valid fix with up to 10 satellites in view under open-sky conditions
Position accuracy (typical)	Consistent to within ~1–2 metres between successive readings while stationary
Data publish interval	15 seconds per update cycle (configurable via publishInterval)
Cloud update latency	Dashboard values and map marker updated within a few seconds of each publish
Reliability	MQTT auto-reconnect logic maintained the cloud link across brief Wi-Fi interruptions
Known limitation	Adafruit IO free-tier rate limiting (throttle warnings) observed at the tested publish rate

Conclusion

The prototype successfully demonstrates a working, low-cost GPS-based vehicle tracking and fleet management system. Live location, speed, and satellite data were reliably captured on the ESP32 and published to a cloud dashboard, giving remote visibility into vehicle status in near real time. Future improvements could include increasing the publish interval to avoid rate-limit throttling, adding an OLED for local status display, integrating the Li-ion/TP4056 power stage for standalone in-vehicle operation, and adding geofencing or historical route-logging features on the dashboard side.